

Context Engineering 기반 AI Agentic Coding System — 모델 중립 공통 계약·Adapter와 비용·품질 균일화

1인 풀스택 서비스(대학생 룸메이트 매칭, 비공개 저장소)를 AI coding agent(OpenAI Codex, Anthropic Claude Code)와 개발하며 설계·구현한 **agent workflow architecture**의 발췌본입니다. 학교 식별 정보는 제거했습니다 (프로젝트명 `Roommate Matching`, 역할 접두어 `repo_`). 제품 코드는 포함하지 않았고 PR 번호는 비공개 저장소 기준입니다.

한 줄 요약: Codex 전용으로 묶여 다른 agent·모델을 쓸 수 없던 workflow를 모델 중립 공통 계약으로 추상화하고 Codex·Claude Code를 adapter로 연결했습니다. 그 위에서 "무엇을 읽히고(Context), 누가 어떤 모델로 하고(Routing), 누가 검증하는가(Multi-agent QA)"를 저장소가 결정하게 해, 도구·모델이 바뀌어도 비용과 품질이 같은 기준을 따르게 했습니다.

1. 문제 — 특정 도구에 묶인 workflow와 agent 결과의 비용·품질 편차

관찰한 문제	결과
workflow가 Codex 전용(공통·중립 계층 없음)	다른 agent·모델(Claude Code 등)을 쓰면 같은 절차·품질 기준이 적용되지 않음. 주간 한도 소진으로 도구를 바꿔야 할 때 실제로 문제가 됨
역할·모델·effort가 개인/전역 설정에 따라 달라짐(저장소에 기본값 없음)	clone·세션마다 같은 요청의 품질이 달라짐
탐색 로그·중간 산출물·전체 대화가 parent context에 누적	context 낭비, 오래된 정보가 최신 코드보다 우선되는 오류
"위험해 보이는 단어"로 effort를 정함(위험도와 추론 난이도가 섞임)	쉬운 작업에 비싼 추론, 어려운 작업에 부족한 추론
모든 child를 가장 비싼 모델로 실행 / 또는 전면 저가 모델	주간 사용량 한도 조기 소진 / 품질 저하 위험
agent의 "완료" 보고·역할 이름·설정 파일 존재를 근거로 판단	실제 모델·권한·검증이 지켜졌는지 알 수 없음

2. 해결 설계

2.1 Context Engineering — 무엇을, 언제, 얼마나 읽힐 것인가

- **신뢰 순서 고정**: 현재 요청 → 현재 코드·계약·테스트 → 정책/설계 문서 → 활성화 계획 → 과거 기록, 과거 대화·메모리는 기본 입력이 아닙니다(`AGENTS.md` §1, 방법론 문서 "Context Engineering").
- **Progressive disclosure / 문서 graph**: 71개 개발 문서를 intent → 권한 → context → 위임 → QA → 인계로 연결된 graph로 감사하고, 작업에 필요한 최소 readset만 읽게 했습니다(`agent-workflows/README.md`, `adapters/README.md`).
- **Scoped packet**: child에게 전체 이력 대신 목적·범위·source 포인터·acceptance·출력 형식·중단 조건만 전달합니다(`request-intake/delegation.md`). 독립 리뷰어에게는 builder의 추론·기대 결론을 주지 않습니다(`qa.md`).
- **Bounded checkpoint**: 긴 작업은 objective / branch·HEAD / 변경 파일 / 검증 / 다음 행동 / blocker만 인계합니다(`session-continuity.md`). 계획은 host 공통 `.plans/` 하나에 두고 Codex↔Claude가 같은 계획을 이어받습니다.

2.2 적응형 모델·effort 라우팅 — 비용을 줄이되 품질 하한을 지킨다

- **두 번 판정:** Pass 1은 요청의 의도·위험, Pass 2는 저장소를 실제로 읽은 뒤의 근거로 역할과 effort를 정합니다. 위험도(impact)와 추론 난이도(complexity)를 분리했습니다(agentic-architecture-decisions.md DA-009).
- **유한 profile:** 검증 가능한 bounded 작업만 저가 모델로 보냅니다(Codex: 구현·조사 Sol, 고정 추출 Luna / 기본·최종 QA Astra). 공식 단가 비율은 Sol/Astra 20%, Luna/Astra 1%이지만 **절감률을 주장하지 않습니다** (비교에는 Parent+child+QA+재작업 전체가 필요, agentic-orchestration-case-study.md "비용 가설과 관측 한계").
- **한 번의 복구:** 저가 모델 실패 시 writer를 회수하고 기본 모델로 한 번만 복구합니다. 권한·quota 문제를 모델 교체로 우회하지 않습니다.
- **Claude 적용:** task profile(haiku 추출 / sonnet 표준 / opus 복잡·독립 리뷰)을 Lead가 고르고, 실행 직전 hook이 요청 모델을 검사하며 실행 후 실제 모델을 기록해 대조합니다.

2.3 Multi-agent architecture — 필요한 때만, 한 명이 쓰고, 다른 한 명이 검증

- **Direct-first:** 결정적 도구 → 단일 agent → 독립 증거가 있을 때만 bounded child(orchestration.md pattern selector).
- **One writer + leader synthesis:** 쓰기 owner는 하나, 결과 통합·판정은 Parent만 합니다. 동시 writer는 lease로 직렬화했습니다.
- **Maker-checker 독립 QA:** fresh context 리뷰어가 요구사항·snapshot·diff만 보고 판정하고, Parent가 finding을 재현한 뒤 판정합니다. PASS는 미실행 검증(notRun)·gap이 있으면 거부됩니다. 반복은 기본 2회로 제한합니다.

2.4 Host-neutral 공통 계약 + Codex/Claude adapter — 도구가 바뀌어도 같은 품질 절차

- 공통 계약(실행·orchestration·QA·위임·계획 lifecycle)이 책임을 소유하고, adapter는 host의 모델·권한·수명주기만 연결합니다.
- Codex도 공통 계약의 소비자로 재연결했고, synthetic 제3 adapter conformance test로 새 host 확장성을 검사합니다.
- 모든 규칙에 강도를 표기합니다: deterministic-static(테스트) / advisory-policy(문서) / model-mediated(agent 판단) / native-runtime(host 강제). 정적 PASS를 실제 모델·권한 입증으로 세지 않습니다.
- 공식 문서 원문 대조로 "hook이 고장 나면 통제가 조용히 풀린다(fail-open)"를 확인하고, 역할별 ask 권한 규칙 + PermissionRequest grant로 재설계해 고장 시 자동 승인 대신 **사람 승인**으로 넘어가게 했습니다.

3. 결과 요약 (상세: EVIDENCE.md)

- 공통 계약 재설계·Codex 재연결 PR 2건 병합, Claude adapter PR CI 11/11 통과(병합 전).
- workflow 테스트 467개, 저장소 정책 테스트 1,116개 통과(최신 작업 트리).
- Codex: bounded 구현·조사·추출을 저가 모델 xhigh로 실행해 각 oracle과 독립 QA 통과(표본 소수, 한계 명시).
- Claude: 잘못된 모델 요청 사전 차단, Sonnet 구현 → Opus 독립 리뷰 → 완료 판정, 독립 리뷰가 근거 없는 주장과 우회 결함 5건 적발.

4. 다음 문제 해결 방향 (설계 단계, ROADMAP.md)

- Claude subagent를 (역할 × 모델 × effort 단계) profile로 재설계: 지금은 child가 Parent effort를 그대로 상속해 비용·품질이 세션마다 달라짐.
- 실패 원인별 복구: 정보가 충분한데 오답 → 상위 모델, 누락·중단 → effort 상향(공식 권고 근거).
- 등록·리뷰 packet 생성 helper로 수작업 비용 제거(수작업이 번거로우면 통제 밖 경로로 우회하게 되는 운영 위험).
- 비용 실측: Parent+child+QA+복구 전체를 같은 task·snapshot에서 비교하는 측정 설계.

5. 재설계 프로그램 — Codex 전용 workflow를 공통 계약 + host adapter로 (H4-G → Q)

처음 만든 workflow는 Codex 전용이어서, 도구를 바꾸면 같은 품질 절차가 적용되지 않았습니다. 기존 구조를 복제하지 않고, **공통 계약을 먼저 만들고 Codex와 Claude를 각각 그 계약의 소비자**로 다시 붙이는 순서로 단계를 나눴습니다. 각 단계는 독립된 PR·완료 기준·rollback을 가집니다.

단계	풀려는 문제	해결 방향	상태
G 공통 계약	공통 문서에 Codex 전제(모델명·effort·도구)가 섞여 있어 다른 host가 따라갈 수 없음	71개 문서를 graph로 감사하고, 31개 파일·341개 host 의존 후보를 문단 단위로 공통/Codex 전용/조건부/역사로 분류. 공통 계약 ·.plans 계획 lifecycle·synthetic 제3 adapter conformance 구현	완료(PR #484 병합)
C Codex 재접합	Codex가 공통 계약을 우회하는 "특권 경로"로 남음	Codex도 공통 계약을 소비하도록 재연결. 기존의 정상 모델/effort/QA/복구 동작은 회귀 oracle로 고정하되 구조·버그는 보존하지 않음(preflight 버그 수정)	완료(PR #485 병합)
L Claude adapter	Claude Code에서 같은 품질 절차(모델 통제·독립 QA)를 강제할 방법이 없음	실제 호출 경로(등록 → hook → 결과 수집 → 공통 QA)에 연결, 공식 문서 대조로 hook fail-open 확인 후 권한 gate 재설계	진행 중(PR #487, CI 통과·병합 전, live 검증 대기)
B 교차 CLI 리뷰	한 host의 리뷰를 다른 host 모델에 맡기는 실제 실행기가 없음(fake process만)	Lead 전용·구독 인증·읽기 전용·단발성 실행기, 실행 전 negative 검증에서 provider 호출 0회 보장	설계
Q 통합 자격 검증	정적 테스트 PASS를 실제 모델·권한·수명주기 동작으로 오인할 위험	양방향 maker/checker, Codex↔Claude 계획 인계, 실패 경로·정리까지 실제 실행으로 판정	설계

공통 원칙: 파일 수 대칭이 아니라 **기능 대등성(capability parity)**을 목표로 하고, 테스트·fixture도 "Codex 가정이 정답으로 남아 있지 않은지" 감사 대상에 넣었습니다. Live 검증은 별도의 신뢰 경계로 두고, 지원되지 않는 모드는 PASS 대신 NO-GO로 기록합니다.

폴더 안내

경로	내용
AGENTS.md, CLAUDE.md	모든 agent 공통 저장소 지침 / Claude 진입점
docs/development/agentic-engineering-methodology.md	채택한 방법론(Context Engineering, Harness Engineering 등)과 적용 수준
docs/development/agentic-orchestration-case-study.md	parent-subagent·모델 라우팅 설계의 문제·대안·실측·한계
docs/development/agent-workflows/	공통 계약과 adapters/ (Codex·Claude·교차 인계)
.codex/, .claude/	역할 정의(TOML/Markdown), Claude hook·권한 설정
scripts/agents/{common,codex,claude,bridge}	계약을 실제로 소비하는 코드(라우팅·profile·등록·hook·collector)
scripts/ci/*.test.mjs	회귀·negative 테스트
portfolio.pdf	이 README와 EVIDENCE·ROADMAP의 PDF 버전

발체본이라 원 저장소의 다른 문서로 가는 일부 상대 링크는 열리지 않습니다.

확인된 결과 (2026-09-28 기준)

"확인됨"은 병합된 PR, CI, 테스트 실행 출력 또는 실제 세션 기록으로 확인한 것만 적었습니다. PR 번호는 비공개 저장소 기준입니다.

1. 통합된 변경

단위	내용	상태
공통 계약 재 설계	71개 개발 문서를 문서 graph로 감사, host-neutral 계약·계획 lifecycle·adapter conformance	PR #484 병합
Codex 재접합	Codex도 공통 계약을 소비하도록 재연결, 기존 모델/QA/복구 동작 회귀 고정, preflight 버그 수정	PR #485 병합
Claude adapter	진입점·task 등록·hook·결과 수집·독립 리뷰·권한 gate	PR #487 진행 중(최종 commit CI 11/11 통과, 병합 전)

2. 테스트

- Claude/Codex/공통 workflow 대상 테스트: **467/467 통과**, 저장소 전체 정책 테스트 **1116/1116 통과** (최신 작업 트리 기준. 병합 전 최종 commit 기준은 1105/1105).
- Negative 테스트 예: 모델 누락·불일치, 다른 세션, 만료, 입력 변조, 재사용, 권한 모드 변경, 독립 리뷰어가 자기 자신을 검토하는 경우, 리뷰 결과를 다른 리뷰어로 바꿔치기하는 경우, 이미 실행된 작업에 뒤늦게 권한을 붙이는 경우.

3. Codex 단계: 비용·품질 라우팅 실측 (2026-09-24, 사전에 고정한 oracle)

작업 profile / 역할 / 모델	결과	한계
bounded-build / executor / Sol xhigh	child 테스트 5/5, Parent oracle 8/8, 독립 QA PASS	구현 표본 1개
bounded-read / explorer / Sol xhigh	고정 source 4개 의미 map, oracle·독립 QA PASS	조사 표본 1개
bounded-extract / explorer / Luna xhigh	변환 8건 oracle 8/8, 독립 QA PASS	task 1개 안의 case 8개
복구(Astra low, 의도적으로 주입한 실패)	복구 oracle 9/9, 독립 QA PASS	실제 실패가 아닌 controlled fault

공식 단가 비율(Sol/Astra 20%, Luna/Astra 1%)은 선택 근거일 뿐, 실제 절감률은 측정하지 않았습니다(출처: case study).

4. 실제 Claude Code 세션에서 관측한 것 (일반 auto 모드)

관측	결과
잘못된 모델·모델 누락 요청	실행 전 hook이 거부, subagent 0개 (여러 회)
입력 파일이 등록 후 바뀐 요청	input file drift로 거부
Sonnet executor 구현 → Opus 독립 reviewer → 공통 QA	완료 판정 통과. 변경 파일 1개(2,900개 경로 대조), writer lease 회수
Sonnet researcher 결과	Opus 독립 reviewer가 공식 문서에 없는 주장(과잉 인용)을 FAIL로 적발, Parent가 원문으로 재현, 완료 거부
Haiku explorer	정확한 모델 ID와 oracle 일치 확인
권한 grant 없이 실행된 subagent	모델·결과가 맞아도 완료 판정 거부(missing-permission-grant)

5. 독립 리뷰가 실제로 잡아낸 결함

- 소스 리뷰 1라운드 FAIL(중간 3건): 리뷰어 "쇼핑"(FAIL 뒤 다른 리뷰어 PASS로 우회), 빈 JSON 결과 통과, 리뷰어가 실행 주체를 검함 → 테스트로 재현 후 수정.
- 2라운드 FAIL(중간 1건): 전달됐지만 기록되지 않은 FAIL 우회 → 수정. 3라운드 PASS.
- 권한 gate 리뷰: 이미 실행된 작업에 나중에 권한을 발급하는 우회 → 재현·수정 후 PASS.

6. 공식 문서 대조로 바뀐 설계

- Claude Code 공식 hooks 문서 확인 결과, hook이 실패하면 호출이 그대로 진행됩니다(fail-open).
- 이를 근거로 "hook만으로 막는다"는 가정을 버렸습니다. 대신 역할별 ask 권한 규칙 + PermissionRequest hook 구조로 바뀌, hook이 고장 나면 자동 승인되지 않고 **사람의 승인 요청으로 넘어가도록**(fail-safe to human) 설계했습니다. 새 세션에서의 실측은 대기 중입니다(ROADMAP 참조).

설계·검증 대기 항목 (구현 완료 아님)

아래는 계획 문서에만 있거나 실측이 끝나지 않은 항목입니다. 완료로 주장하지 않습니다.

항목	상태	내용
권한 gate 실측	구현·리뷰 완료, live 검증 대기	정상 / hook 부재 / timeout / 잘못된 출력 / deny 규칙의 7개 시나리오를 새 세션에서 실행해 "고장 시 사람 승인으로 넘어가는지" 확인
Claude subagent 아키텍처 재설계	설계	모델과 effort를 함께 고르는 profile(역할 × 모델 × effort 단계), 관측된 effort와 기대값 대조, 실패 원인별 복구(정보 충분한데 오답이면 상위 모델, 누락·중단이면 effort 상향; 공식 권고 근거)
운영 helper	설계	수작업 JSON 없이 task 등록·독립 리뷰 packet을 만드는 명령, Parent 동시 쓰기 차단
공통 지침 누락 점검	설계	Codex 문서에만 있는 일반 원칙(복구 한도, 품질 stop 등)을 공통 계약으로 승격
교차 host 리뷰 실행기	설계(현재 fake process만)	Claude Lead ↔ Codex CLI 단발성 읽기 전용 리뷰, 인증·권한·정리 검증
비용 실측 설계	설계	같은 task·snapshot에서 Parent+child+QA+재작업+복구 전체 사용량을 비교(단가 비율이 아닌 총량)
통합 live 자격 검증	설계	양방향 maker/checker, 계획 인계(Codex ↔ Claude), OMC/OMX 수명주기

알려진 한계: hook 기반 통제는 사람이 승인 요청을 수락하면 막지 못합니다(대신 완료 판정에서 거부). 생산성이나 품질 향상 수치는 측정하지 않았습니다.